Problem Statement : Medical Insurence Premium

Import Libraries

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Load Data

In [2]:
```
data = pd.read_csv('/content/insurance.csv')
```

In [3]:
```
data.head()
```

Out[3]:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

Data Inspection

In [4]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   age       1338 non-null   int64  
 1   sex       1338 non-null   object 
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64  
 4   smoker    1338 non-null   object 
 5   region    1338 non-null   object 
 6   charges   1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

In [6]:
```
data.describe()
```

Out[6]:

	age	bmi	children	charges
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.663397	1.094918	13270.422265
std	14.049960	6.098187	1.205493	12110.011237
min	18.000000	15.960000	0.000000	1121.873900
25%	27.000000	26.296250	0.000000	4740.287150
50%	39.000000	30.400000	1.000000	9382.033000
75%	51.000000	34.693750	2.000000	16639.912515
max	64.000000	53.130000	5.000000	63770.428010

In [7]:
```
data.shape
```

Out[7]:
```
(1338, 7)
```

In [8]:
```
data.isnull().sum()
```

Out[8]:

	0
age	0
sex	0
bmi	0
children	0
smoker	0
region	0
charges	0

dtype: int64

In [9]:
```
data.duplicated().sum()
```

Out[9]:
```
1
```

EDA

Univariate Analysis

Numerical Columns

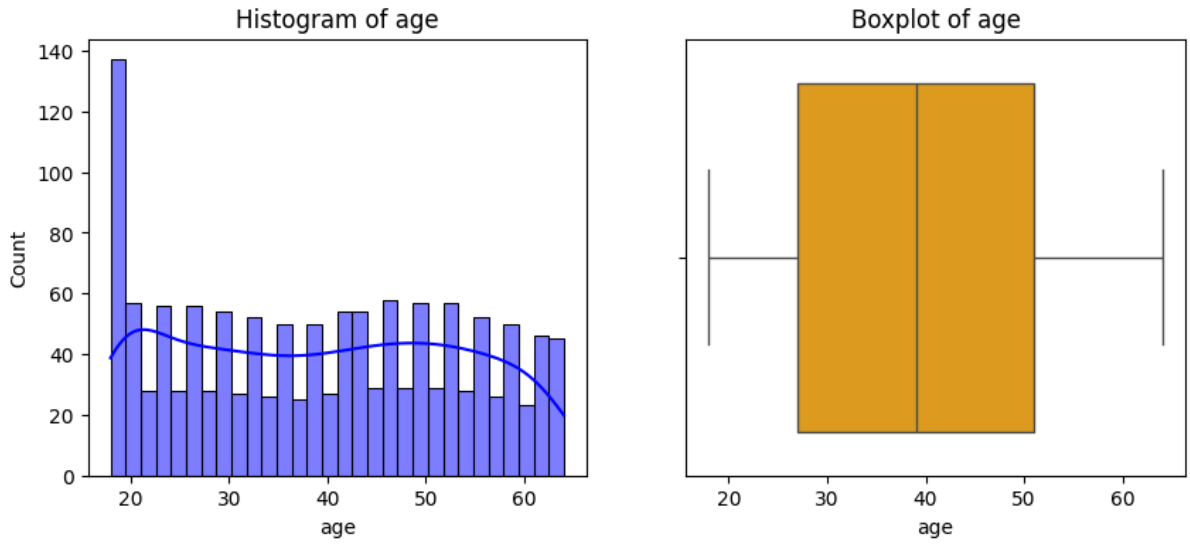
In [29]:
```
# Plot histograms and boxplots
numeric_features = ['age', 'bmi','charges']

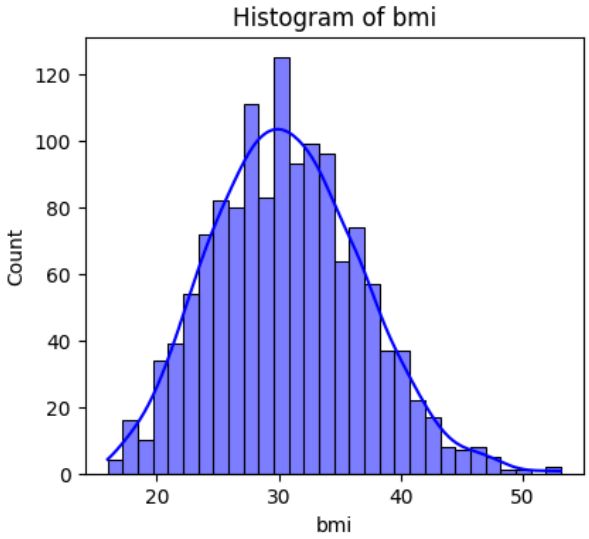
for feature in numeric_features:
    plt.figure(figsize=(10, 4))

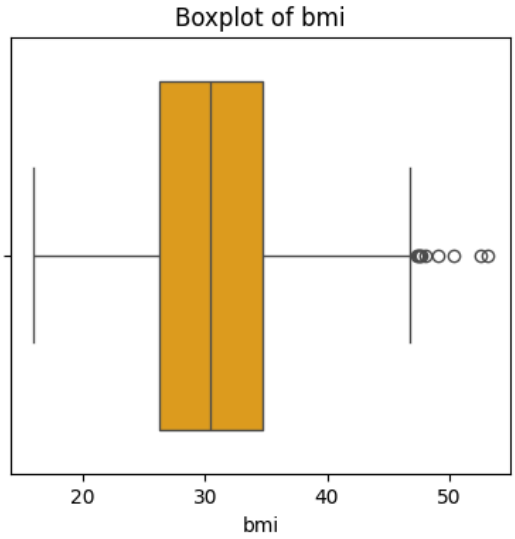
    # Histogram
    plt.subplot(1, 2, 1)
    sns.histplot(data[feature], kde=True, bins=30, color='blue')
    plt.title(f'Histogram of {feature}')

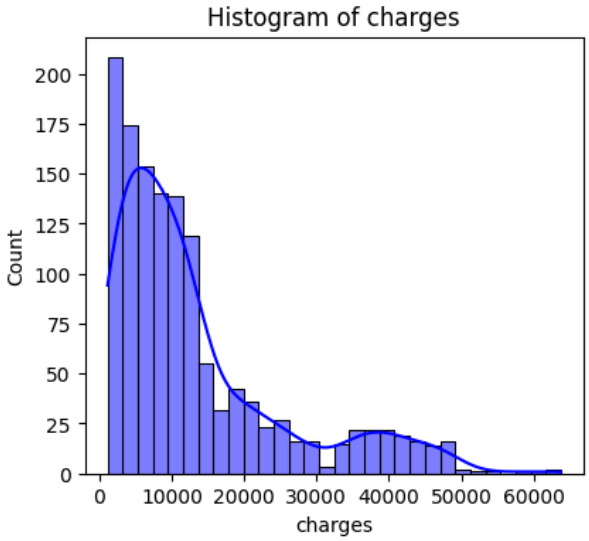
    # Boxplot
    plt.subplot(1, 2, 2)
    sns.boxplot(x=data[feature], color='orange')
    plt.title(f'Boxplot of {feature}')

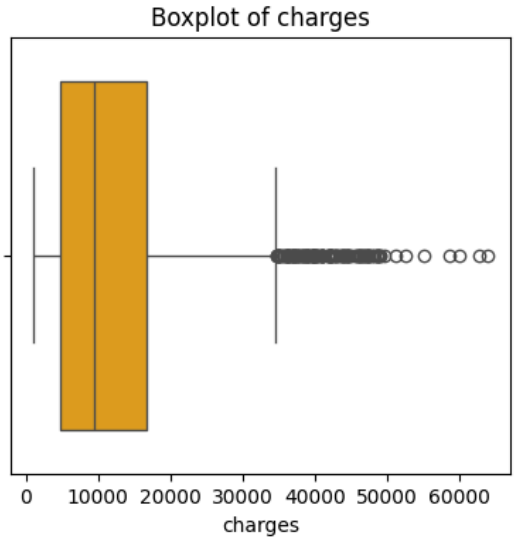
    plt.show()
```




Histogram of bmi
Count
bmi
0
20
40
60
80
100
120
20
30
40
50


Boxplot of bmi
bmi
20
30
40
50

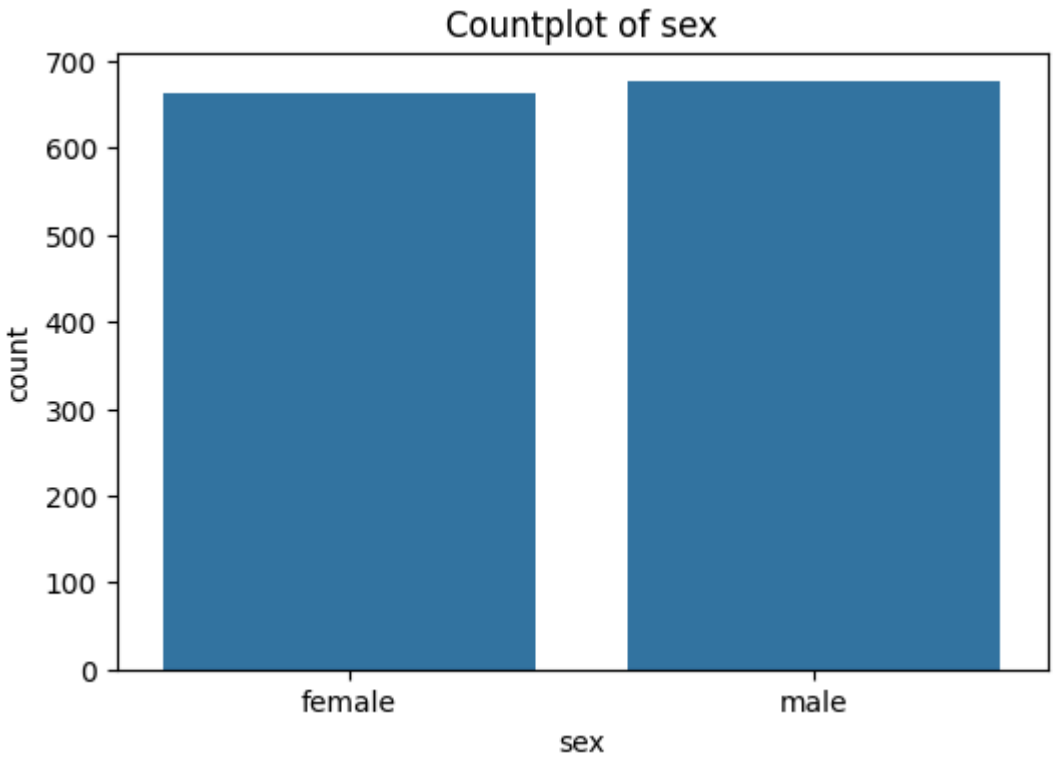

Histogram of charges
Count
charges
0
25
50
75
100
125
150
175
200
0
10000
20000
30000
40000
50000
60000

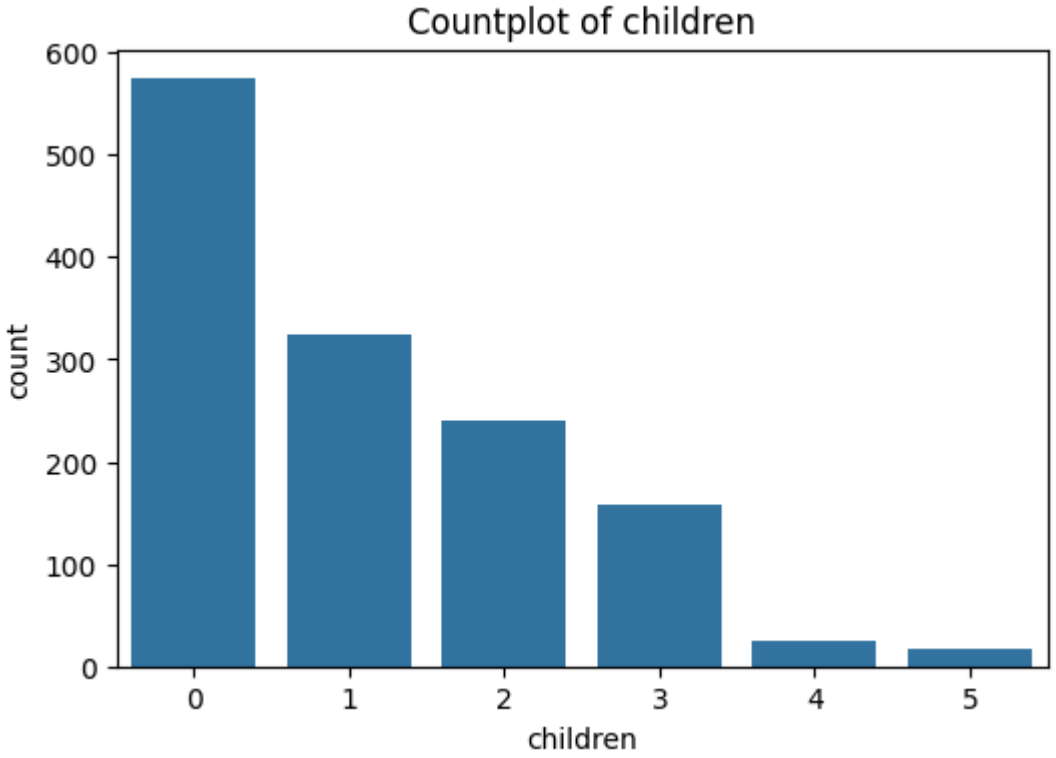

Boxplot of charges
charges
0
10000
20000
30000
40000
50000
60000

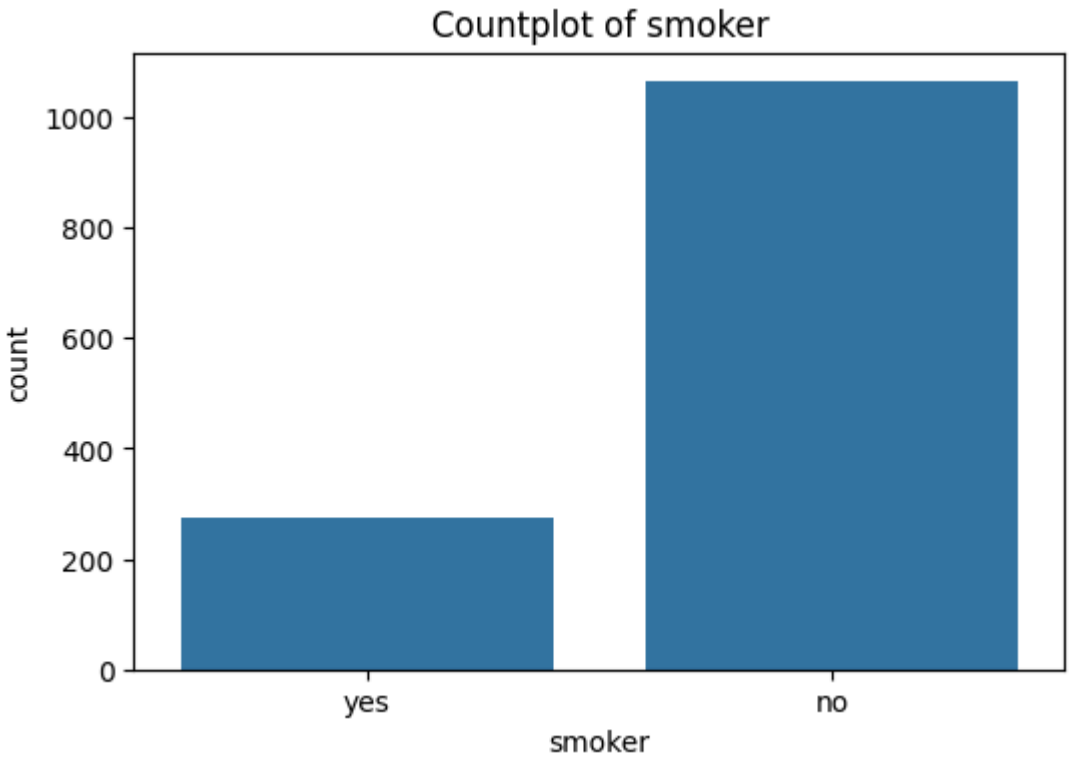
Categorical Columns

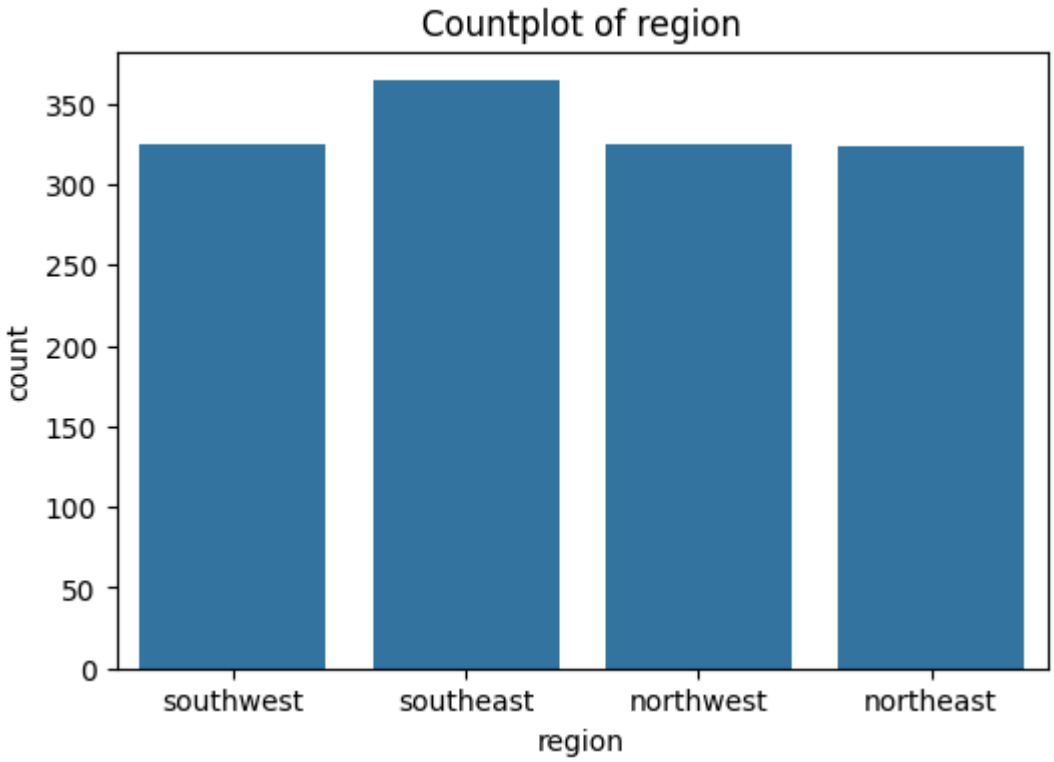
In [31]:
```
# Plot bar plots
categorical_features = ['sex','children', 'smoker', 'region']

for feature in categorical_features:
    plt.figure(figsize=(6, 4))
    sns.countplot(data=data, x=feature)
    plt.title(f'Countplot of {feature}')
    plt.show()
```




Countplot of children
600
500
400
300
200
100
0
count
0
1
2
3
4
5
children


Countplot of smoker
1000
800
600
400
200
0
count
yes
no
smoker


Countplot of region
count
0
50
100
150
200
250
300
350
southwest
southeast
northwest
northeast
region

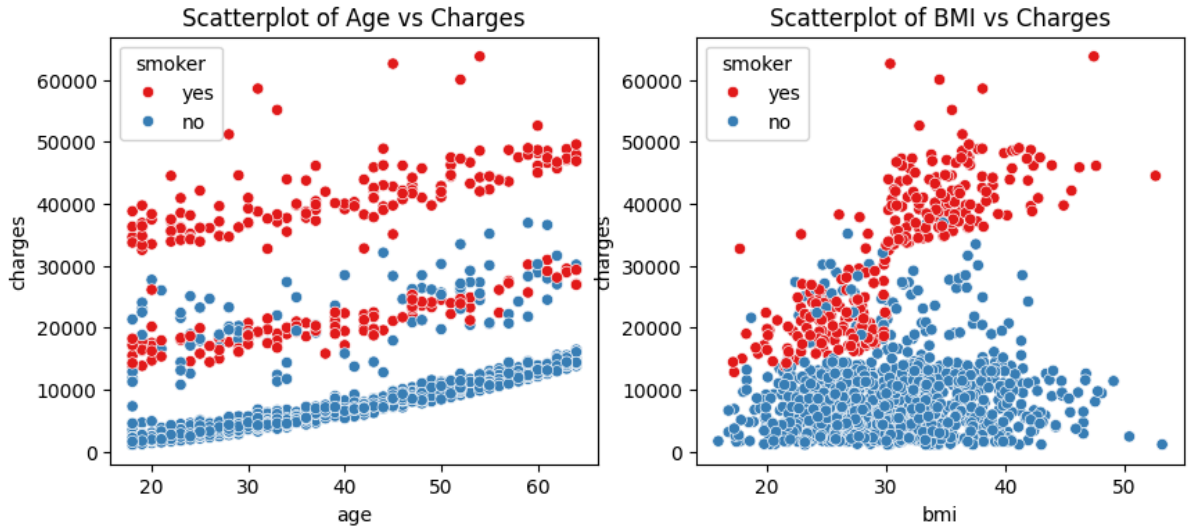
Bivariate Analysis

In [36]:
```
# Scatterplots for charges vs other numeric features
plt.figure(figsize=(10, 4))

plt.subplot(1,2,1)
sns.scatterplot(data=data, x='age', y='charges', hue='smoker',
palette='Set1')
plt.title(f'Scatterplot of Age vs Charges')

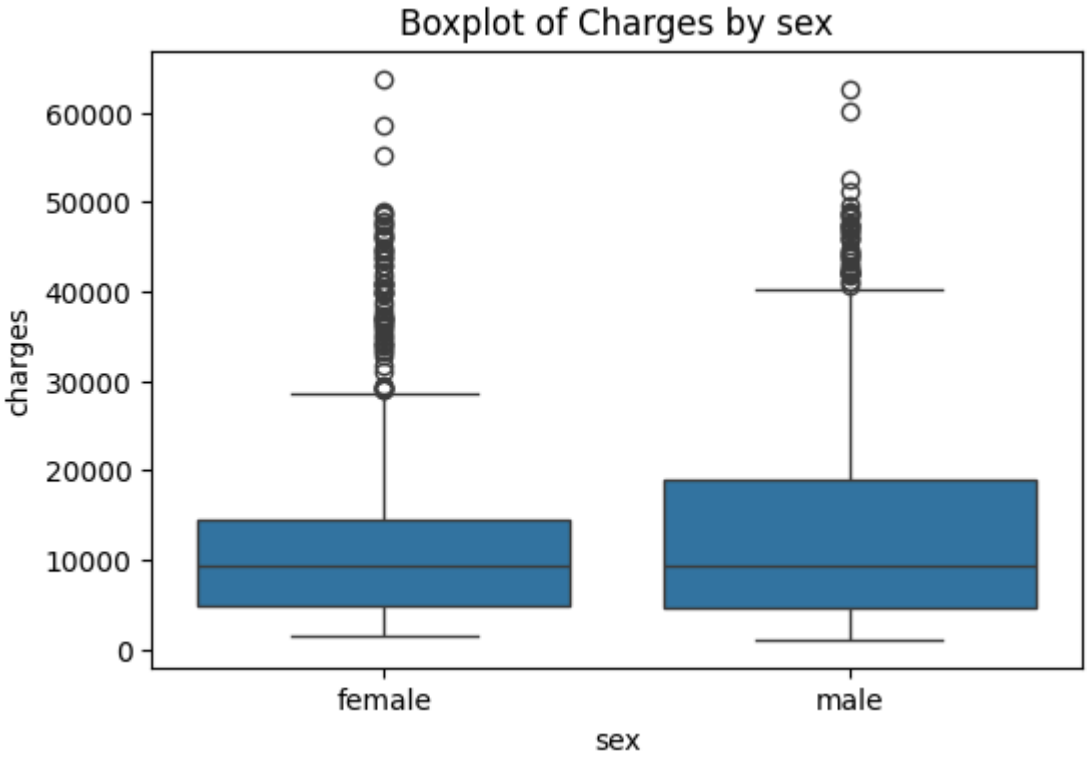
plt.subplot(1,2,2)
sns.scatterplot(data=data, x='bmi', y='charges', hue='smoker',
palette='Set1')
plt.title(f'Scatterplot of BMI vs Charges')

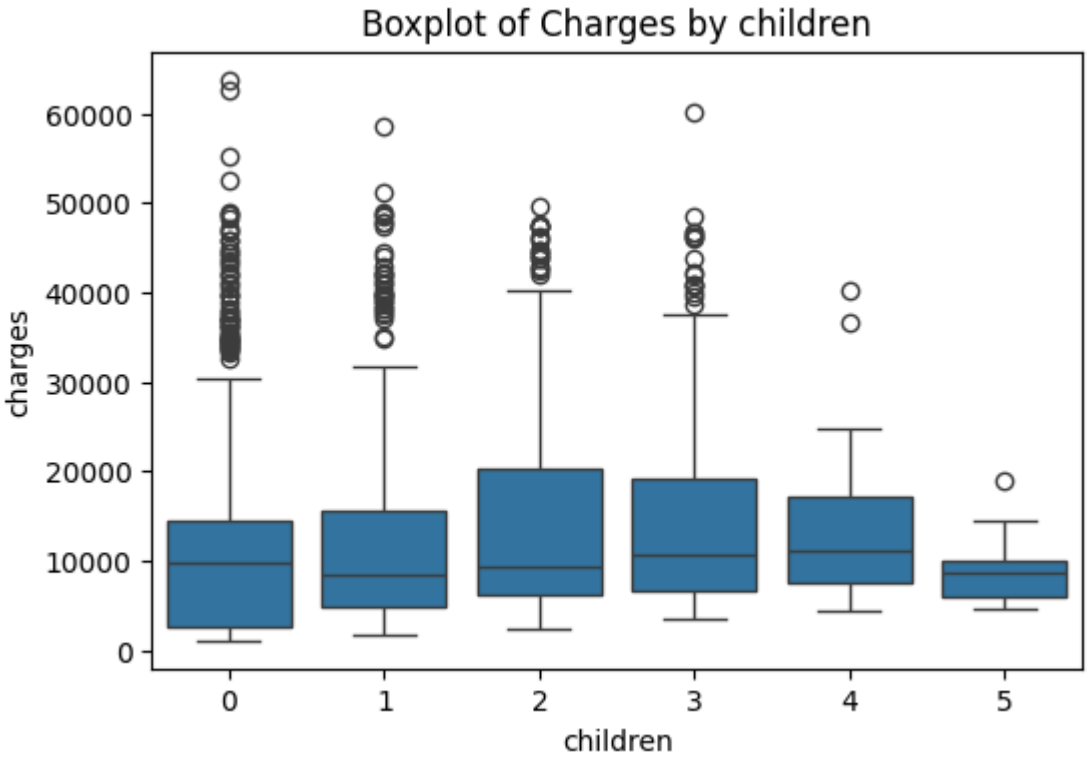
plt.show()
```

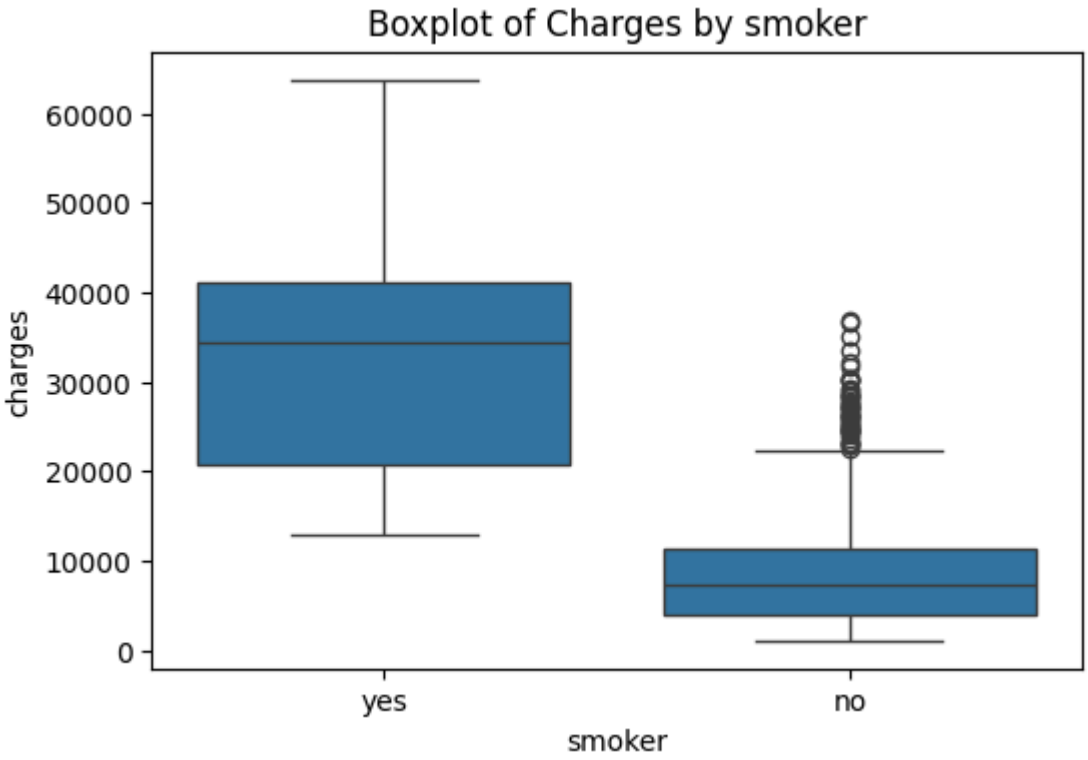


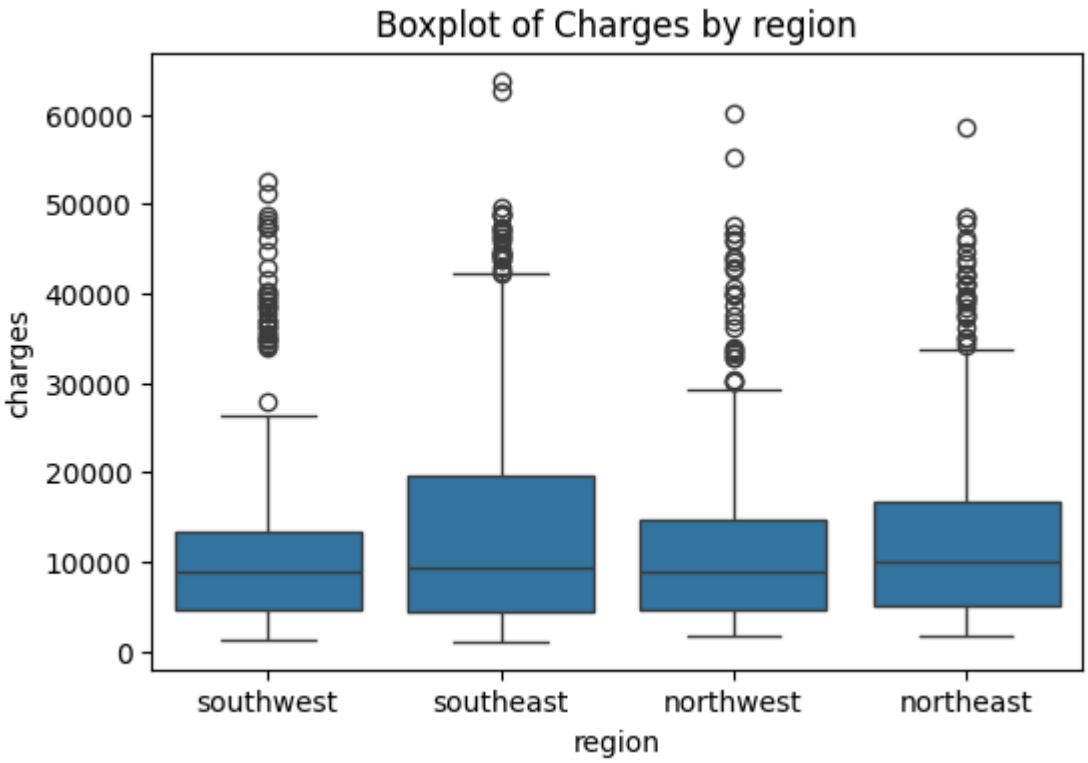
In [38]:
```
# Boxplots for charges across categories
categorical_features = ['sex','children','smoker', 'region']

for feature in categorical_features:
    plt.figure(figsize=(6, 4))
    sns.boxplot(data=data, x=feature, y='charges')
    plt.title(f'Boxplot of Charges by {feature}')
    plt.show()
```









One hot encoding

In [39]:
```
from sklearn.preprocessing import OneHotEncoder
```

In [80]:
```
data.head()
```

Out[80]:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

In [81]:
```
# apply one hot encoding and replace data
encoded_df = pd.get_dummies(data, columns=['sex', 'smoker', 'region'],
drop_first=True).astype(int)
encoded_df.head()
```

Out[81]: Columns 1–8 of 10

	age	bmi	children	charges	sex_male	smoker_yes	region_northwest
0	19	27	0	16884	0	1	0
1	18	33	1	1725	1	0	0
2	28	33	3	4449	1	0	0
3	33	22	0	21984	1	0	1
4	32	28	0	3866	1	0	1

Columns 9–10 of 10 · first column repeated

	region_southeast	region_southwest
0	0	1
1	1	0
2	1	0
3	0	0
4	0	0

Split Data

In [82]:
```
X = encoded_df.drop(['charges'], axis = 1)
y = encoded_df['charges']
```

In [83]:
```
X
```

Out[83]:

Columns 1–8 of 9

	age	bmi	children	sex_male	smoker_yes	region_northwest	region_southeast
0	19	27	0	0	1	0	0
1	18	33	1	1	0	0	1
2	28	33	3	1	0	0	1
3	33	22	0	1	0	1	0
4	32	28	0	1	0	1	0
...	...	...	...	...	...	...	...
1333	50	30	3	1	0	1	0
1334	18	31	0	0	0	0	0
1335	18	36	0	0	0	0	1
1336	21	25	0	0	0	0	0
1337	61	29	0	0	1	1	0

Columns 9–9 of 9 · first column repeated

	region_southwest
0	1
1	0
2	0
3	0
4	0
...	...
1333	0
1334	0
1335	0
1336	1
1337	0

1338 rows × 8 columns

In [84]:
```
y
```

Out[84]:

	charges
0	16884
1	1725
2	4449
3	21984
4	3866
...	...
1333	10600
1334	2205
1335	1629
1336	2007
1337	29141

1338 rows × 1 columns

dtype: int64

In [85]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1,
random_state=42)
```

In [86]:
```
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(1204, 8)
(134, 8)
(1204,)
(134,)
```

Build Multiple Linear Regression Model

In [87]:
```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

lr = LinearRegression()
lr.fit(X_train, y_train)
```

Out[87]:
```
LinearRegression()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

LinearRegression?Documentation for LinearRegressioniFitted

```
LinearRegression()
```

In [88]:
```
y_pred = lr.predict(X_test)
```

In [89]:
```
mse = mean_squared_error(y_test, y_pred)
print(f'Mean Squared Error: {mse}')
```

```
Mean Squared Error: 32201893.448008858
```

The `Mean Squared Error (MSE)` value of 32,201,893.45 indicates the average squared difference between actual and predicted insurance charges. A large MSE suggests the model might be underperforming or not well-tuned.

Steps to Improve the Model:

1. `Feature Engineering`: Check for missing important variables or transformations (e.g. log transformation).
2. `Data Scaling`: Ensure numerical features like age, bmi, etc., are scaled properly.
3. `Outlier Treatment`: Remove or handle extreme values that may skew predictions.
4. Try feature selection or regularization.

In [91]:
```
encoded_df['charges'] = np.log(encoded_df['charges'])
encoded_df.head()
```

Out[91]: Columns 1–8 of 10

	age	bmi	children	charges	sex_male	smoker_yes	region_northwest
0	19	27	0	2.275637	0	1	0
1	18	33	1	2.008614	1	0	0
2	28	33	3	2.128283	1	0	0
3	33	22	0	2.302392	1	0	1
4	32	28	0	2.111422	1	0	1

Columns 9–10 of 10 · first column repeated

	region_southeast	region_southwest
0	0	1
1	1	0
2	1	0
3	0	0
4	0	0

In [92]:
```
X = encoded_df.drop(['charges'], axis = 1)
y = encoded_df['charges']
```

In [93]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

In [94]:
```
lr = LinearRegression()
lr.fit(X_train, y_train)
```

Out[94]:
```
LinearRegression()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

☑ LinearRegression?Documentation for LinearRegressioniFitted

```
LinearRegression()
```

In [95]:
```
y_pred = lr.predict(X_test)
```

In [96]:
```
y_pred
```

Out[96]:
```
array([2.21028462, 2.15222177, 2.42450455, 2.2096227 , 2.24321207,
       2.15466879, 2.07375345, 2.26357362, 2.10917179, 2.22564516,
       2.30334108, 2.18529149, 2.12435778, 2.38945116, 2.40834363,
       2.37662214, 2.23290985, 2.37199217, 2.19050045, 2.34110713,
       2.13819526, 2.18803763, 2.06734276, 2.1154733 , 2.23904778,
       2.23296847, 2.24644518, 2.15044679, 2.21959675, 2.06128936,
       2.204091  , 2.2369887 , 2.08811266, 2.14729603, 2.10646415,
       2.19738201, 2.09056088, 2.18203815, 2.38543211, 2.31750698,
       2.13064632, 2.09988395, 2.24188221, 2.2272829 , 2.15035103,
       2.23340869, 2.11940525, 2.12635158, 2.36795063, 2.14651697,
       2.25764652, 2.0712421 , 2.19449392, 2.07235306, 2.21767703,
       2.23029644, 2.11863921, 2.32894304, 2.23576354, 2.23036668,
       2.25288103, 2.15882456, 2.27632145, 2.19047702, 2.22228683,
       2.124926  , 2.2802911 , 2.22480245, 2.10890354, 2.09158232,
       2.1781661 , 2.22384254, 2.19396839, 2.17007032, 2.18682168,
       2.14083466, 2.15027867, 2.2248003 , 2.13718679, 2.2071472 ,
       2.07334052, 2.41365479, 2.14967604, 2.30454486, 2.28519047,
       2.3728467 , 2.14428131, 2.22098429, 2.20266542, 2.24455101,
       2.29560052, 2.40900889, 2.37483878, 2.14799406, 2.30559306,
       2.18009502, 2.29203515, 2.08277183, 2.31933838, 2.16783253,
       2.13697716, 2.07403504, 2.16632454, 2.25646492, 2.23901065,
       2.07945919, 2.18718781, 2.35315418, 2.06550548, 2.37664678,
       2.07145131, 2.12629431, 2.26017595, 2.33005939, 2.22169517,
       2.08674487, 2.24188692, 2.35581614, 2.16881999, 2.09837719,
       2.16623413, 2.19076956, 2.25588746, 2.09196477, 2.13350923,
       2.19883136, 2.18817315, 2.21089433, 2.24039178, 2.09621056,
       2.12357992, 2.16824638, 2.16772496, 2.20544381, 2.15893922,
       2.24710384, 2.14424829, 2.32524707, 2.36151263, 2.32928153,
       2.14674371, 2.22242407, 2.09990738, 2.24799378, 2.08224092,
       2.36884314, 2.1494125 , 2.12306022, 2.25397521, 2.1422059 ,
       2.42848211, 2.09785577, 2.07635403, 2.31542449, 2.1851818 ,
       2.14685007, 2.2553076 , 2.19584127, 2.24536482, 2.34582542,
       2.27012869, 2.08918425, 2.26184844, 2.08947056, 2.11993378,
       2.18883181, 2.42766241, 2.40355299, 2.36034113, 2.10060191,
       2.19490985, 2.17026147, 2.17766273, 2.1274086 , 2.08457944,
       2.34749618, 2.2297985 , 2.25766457, 2.30774164, 2.22603258,
       2.42842693, 2.09132539, 2.1827324 , 2.15177446, 2.14452612,
       2.10867465, 2.13146822, 2.11634989, 2.21775353, 2.21413671,
       2.12678136, 2.08589292, 2.09356105, 2.31242432, 2.24698293,
       2.20725182, 2.08781474, 2.25013492, 2.08178438, 2.19303697,
       2.10155259, 2.36878708, 2.15076276, 2.09381423, 2.25107177,
       2.26480962, 2.19291067, 2.11826462, 2.19412368, 2.09822901,
       2.22578325, 2.22733723, 2.24575479, 2.25551554, 2.1630855 ,
       2.13080956, 2.13887348, 2.26340242, 2.22929841, 2.15503319,
       2.09949977, 2.18688654, 2.19262318, 2.36073692, 2.08993121,
       2.26893622, 2.06307892, 2.07527826, 2.2036647 , 2.25418484,
       2.08142037, 2.19535716, 2.13091455, 2.32510983, 2.19932847,
       2.19261692, 2.12572191, 2.16416085, 2.36893596, 2.08128313,
       2.23762308, 2.30311219, 2.11401547, 2.11244386, 2.06843068,
       2.10444692, 2.13105179, 2.1316615 , 2.28426859, 2.05747456,
       2.07937586, 2.19155452, 2.10566825, 2.23118723, 2.09740135,
       2.1125811 , 2.22742725, 2.1185082 , 2.18998547, 2.17285618,
```

```
       2.19933473, 2.23917877, 2.26222588, 2.41340997, 2.23548195,
       2.16584046, 2.40353373, 2.22065617])
```

In [97]:
```
y_test
```

Out[97]:

	charges
764	2.209974
887	2.148287
890	2.330819
1293	2.212428
259	2.344373
...	...
109	2.375749
575	2.241878
535	2.164543
543	2.403610
846	2.218927

268 rows × 1 columns

dtype: float64

In [98]:
```
mse = mean_squared_error(y_test, y_pred)
print(f'Mean Squared Error: {mse}')
```

```
Mean Squared Error: 0.0022709319776005347
```

In [99]:
```
y_pred_original = np.exp(y_pred)
y_pred_original
```

Out[99]:
```
array([ 9.11831128,  8.6039532 , 11.29663111,  9.11227765,  9.42355179,
        8.62503297,  7.95462446,  9.61739674,  8.24141284,  9.25945471,
       10.0075627 ,  8.89324049,  8.36752193, 10.90750577, 11.1155345 ,
       10.76846701,  9.32696666, 10.71872457,  8.93968589, 10.39273635,
        8.48411216,  8.91769613,  7.9037929 ,  8.29351019,  9.38439105,
        9.32751346,  9.45406851,  8.58869486,  9.20361873,  7.85609261,
        9.06201041,  9.36508772,  8.06967053,  8.56167656,  8.21912826,
        9.00141704,  8.08945109,  8.86435471, 10.86375596, 10.15033774,
        8.42030726,  8.16522228,  9.41102818,  9.27463173,  8.58787248,
        9.33162049,  8.32618399,  8.38422177, 10.6754918 ,  8.55500907,
        9.56056205,  7.93467265,  8.97545763,  7.94349267,  9.18596734,
        9.30262336,  8.31980828, 10.26708394,  9.35362095,  9.30327678,
        9.51510972,  8.66095122,  9.74078246,  8.93947641,  9.22841053,
        8.37227795,  9.77952681,  9.25165495,  8.23920238,  8.09771819,
        8.83009788,  9.24277843,  8.97074195,  8.75889991,  8.90685926,
        8.50653472,  8.58725108,  9.25163509,  8.47556051,  9.08974818,
        7.95134039, 11.17472789,  8.58207771, 10.01961689,  9.82755791,
       10.72788797,  8.53590433,  9.21639802,  9.04910104,  9.43617781,
        9.93039761, 11.12293163, 10.74928004,  8.56765494, 10.03012499,
        8.84714689,  9.8950551 ,  8.02668675, 10.16894412,  8.73932132,
        8.473784  ,  7.95686472,  8.72615243,  9.54927201,  9.38404259,
        8.00014121,  8.91012092, 10.51869533,  7.88928478, 10.76873237,
        7.93633281,  8.38374163,  9.58477543, 10.27855193,  9.22295206,
        8.05864047,  9.41107251, 10.54673298,  8.74795528,  8.15292853,
        8.72536351,  8.94209191,  9.54375929,  8.10081579,  8.44444837,
        9.01447264,  8.91890469,  9.12387253,  9.39701212,  8.13528326,
        8.36101575,  8.74293881,  8.73838123,  9.07427797,  8.66194433,
        9.46029762,  8.53562247, 10.22920713, 10.60698375, 10.27055982,
        8.55694906,  9.22967714,  8.16541361,  9.46872043,  8.02242639,
       10.68502408,  8.57981629,  8.35667169,  9.52552668,  8.51820725,
       11.34165365,  8.14867851,  7.97533802, 10.12922177,  8.89226498,
        8.55785928,  9.53822679,  8.98755888,  9.4438602 , 10.4418881 ,
        9.6806465 ,  8.07832262,  9.60081929,  8.0806358 ,  8.3305858 ,
        8.9247812 , 11.33236073, 11.06241128, 10.59456491,  8.17108667,
        8.97919157,  8.76057438,  8.82565418,  8.39308879,  8.04120899,
       10.45934861,  9.29799233,  9.56073464, 10.05169868,  9.26304267,
       11.34102778,  8.09563795,  8.87051094,  8.60010541,  8.53799433,
        8.23731674,  8.42723079,  8.30078333,  9.18667006,  9.15350359,
        8.3878259 ,  8.0517779 ,  8.11375729, 10.09887788,  9.45915378,
        9.09069915,  8.0672668 ,  9.48901598,  8.01876466,  8.9623903 ,
        8.17885848, 10.6844251 ,  8.59140905,  8.11581174,  9.49790992,
        9.62929118,  8.96125845,  8.31669232,  8.97213516,  8.15172047,
        9.26073346,  9.27513566,  9.44754376,  9.54021043,  8.69793374,
        8.4216819 ,  8.48986825,  9.61575038,  9.29334369,  8.62817652,
        8.16208595,  8.90743696,  8.95868257, 10.59875902,  8.08435902,
        9.66910951,  7.87016413,  7.966763  ,  9.0581481 ,  9.52752369,
        8.0158463 ,  8.98320897,  8.42256611, 10.22780335,  9.01895497,
        8.95862651,  8.37894416,  8.70729211, 10.68601591,  8.01474627,
        9.37103062, 10.00527236,  8.28142847,  8.26842347,  7.91239631,
        8.20256505,  8.42372212,  8.42885974,  9.81850225,  7.82618026,
        7.9994746 ,  8.94911387,  8.2125892 ,  9.3109137 ,  8.14497644,
        8.26955833,  9.27597062,  8.31871839,  8.93508327,  8.78333501,
```

```
9.0190114 ,  9.38562034,  9.60444373, 11.17199245,  9.35098743,
8.72192925, 11.0621982 ,  9.21337441])
```